

Lasso Regression

Study Material · Theory + Code, block by block

L1-regularised linear regression that selects features

Python

pandas

NumPy

scikit-learn

DIGITAL GYAAN

What is Lasso Regression?

Lasso is **linear regression with an L1 penalty**. It shrinks weights like Ridge, but it can set some weights to **exactly zero** — so it picks features for you.

- Penalty = the sum of **absolute** weights (L1).
- Unimportant weights are driven to **exactly 0** → those features are **dropped**.
- Built-in **feature selection** → simpler, sparser models.

The problem it solves

When you have **many features** and suspect a lot are just noise, you want a model that **ignores the useless ones**.

- Ordinary regression keeps **every** feature — noisy and hard to read.
- Ridge shrinks weights but **still keeps them all**.
- Lasso zeroes the weak ones → **automatic selection**.
- Result: fewer features and a much **clearer story**.

The maths

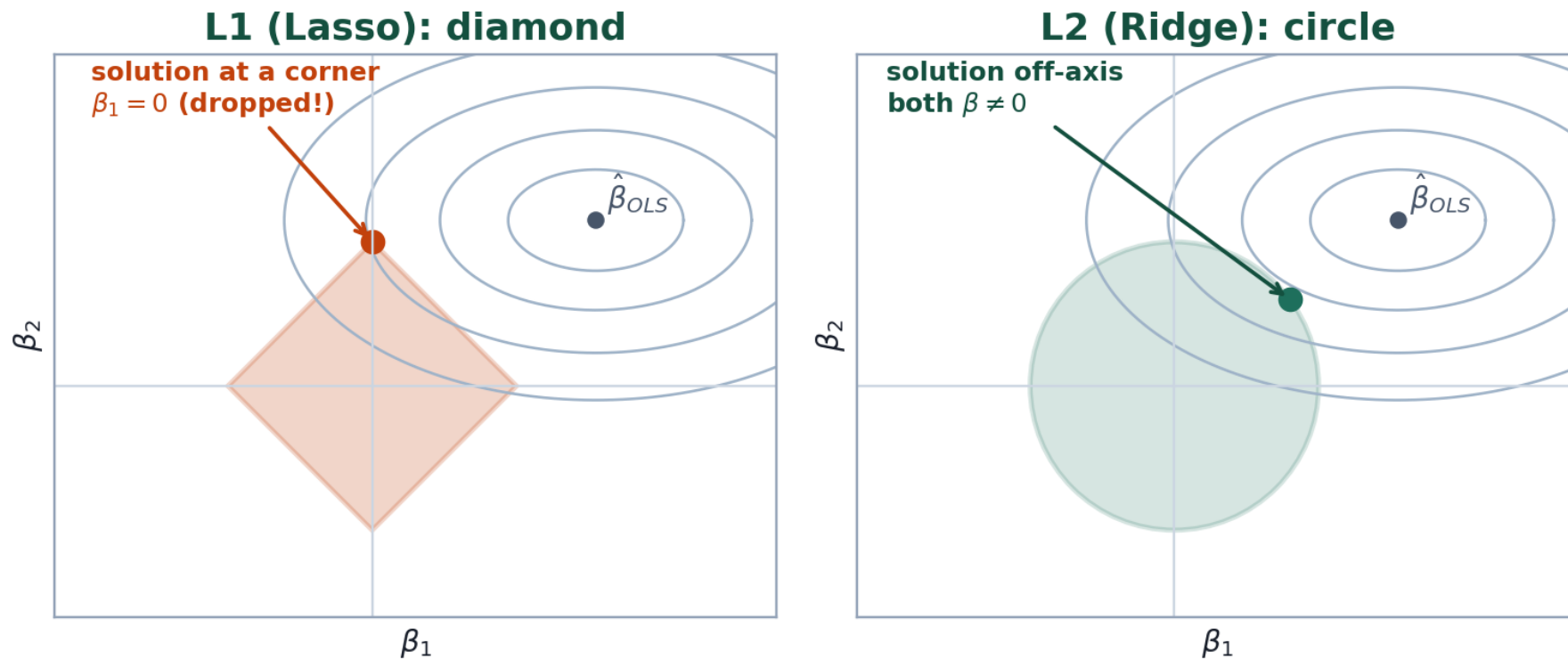
Lasso picks the weights $\mathbf{w}$ that minimise:

$$\text{Cost} = \sum (y_i - \hat{y}_i)^2 + \alpha \sum |w_j|$$

- $\sum (y_i - \hat{y}_i)^2$ = residual sum of squares (fit the data).
- $\alpha \sum |w_j|$ = the L1 penalty (the *absolute* weights).
- The absolute value — not a square — is what lets weights hit **exactly zero**.

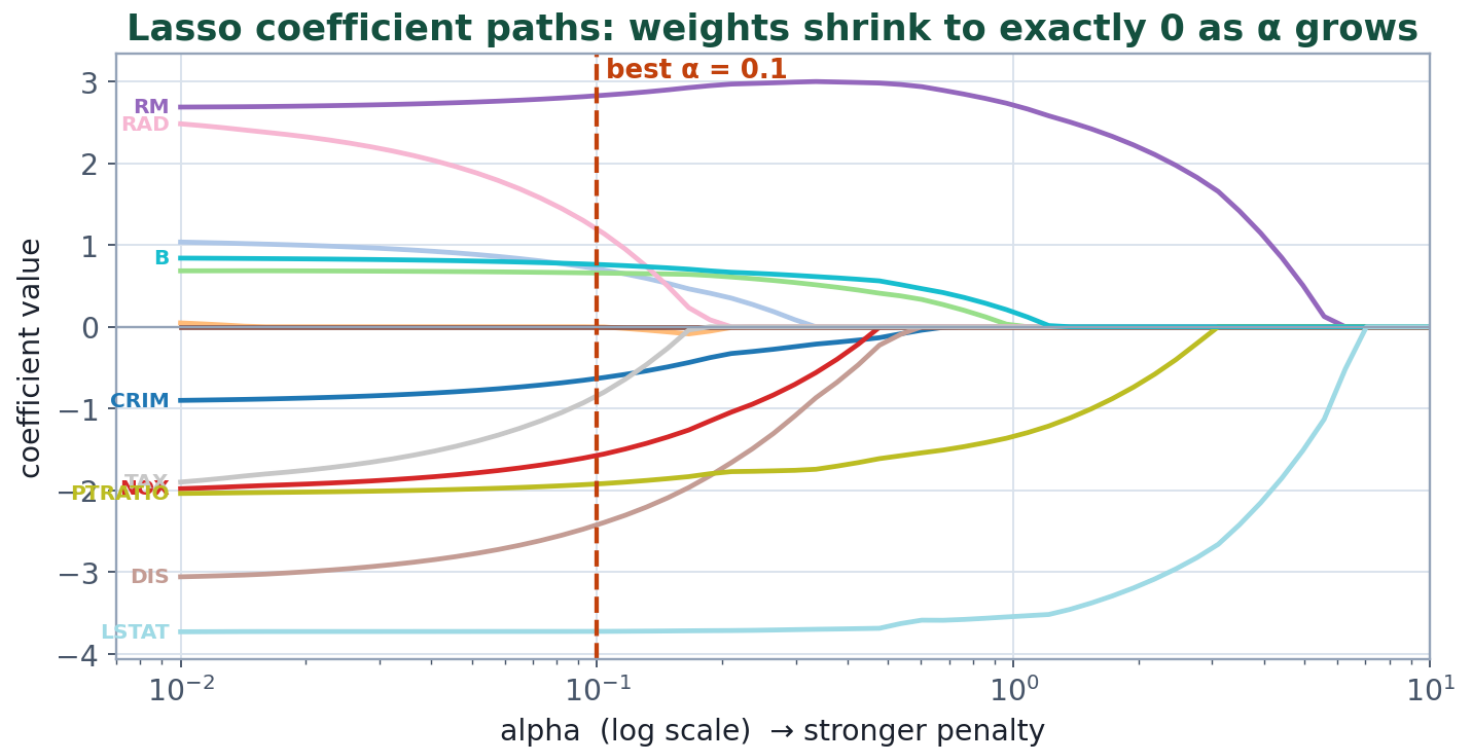
Why L1 sets weights to zero

The L1 region is a **diamond** with sharp corners on the axes; the best solution often lands **on a corner**, forcing a weight to 0. The L2 circle is smooth, so Ridge shrinks but rarely zeroes.



The role of alpha (α)

As **alpha** grows, the penalty gets stronger and **more coefficients snap to exactly zero**. Cross-validation finds the sweet spot — here $\alpha = 0.1$.



Scaling is essential for Lasso

Lasso is **sensitive to feature scale**. A feature in the thousands would be penalised differently from one in single digits — unfair to the small ones.

- Standardise every column to **mean 0, std 1** with StandardScaler.
- Always do it **before** fitting.
- Skip it and your feature selection becomes **misleading**.

Key properties to remember

- Performs **feature selection** — sets weak weights to exactly 0.
- Produces **sparse, interpretable** models.
- With correlated features it tends to **keep one and drop the rest**.
- **Scale-sensitive** — always standardise first.
- Needs **alpha tuning** via cross-validation.

When should you use Lasso?

Reach for Lasso when you have **many features** and believe **only a few truly matter**.

Strengths

- Automatic feature selection
- Sparse, easy-to-read models
- Great for high-dimensional data

Trade-offs

- Unstable with correlated features
- Needs scaling + alpha tuning
- May drop useful-but-weak features

Code Walkthrough

The Lasso script, explained one block at a time

Python

pandas

NumPy

scikit-learn

Import the libraries

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_openml
```

What this block does

- **pandas** — handles the data as a table.
- **numpy** — fast numerical / array math.
- **fetch_openml** — downloads the dataset from openml.org.

Load the dataset

```
boston = fetch_openml(name="boston",  
                      version=1, as_frame=True,  
                      parser="pandas")  
df = boston.frame  
df["Price"] = boston.target
```

What this block does

- Downloads the **Boston housing** dataset.
- df = the table of features.
- We add the value to predict as a column called **Price**.

Split inputs (X) and answer (Y)

```
X = df.drop(columns=["MEDV", "Price"])
Y = df["Price"]
```

What this block does

- X = the features the model learns from.
- Y = the answer (the price).
- Drop **both** price columns from X so the model can't "cheat".

Scale the features (Lasso-specific!)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

What this block does

- Rescales every column to **mean 0, std 1**.
- **Crucial** for Lasso — it is scale-sensitive.
- Returns a NumPy array, ready for fitting.

Build the model + list alphas

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV

lasso = Lasso(max_iter=10000)
params = {"alpha": [1e-3, 1e-2, 0.1,
                    1, 5, 10, 20]}
```

What this block does

- Create a **Lasso** model.
- **max_iter** = enough steps for it to converge.
- params lists the **alphas** to try.

Search + cross-validate

```
lasso_cv = GridSearchCV(lasso, params,  
    scoring="neg_mean_squared_error",  
    cv=5)  
lasso_cv.fit(X, Y)
```

What this block does

- **GridSearchCV** trains once per alpha.
- **cv=5** — 5-fold cross-validation.
- **scoring** — lower error wins (negative, closer to 0).
- **.fit()** runs the search.

Show the results

```
print(lasso_cv.best_params_)  
print(lasso_cv.best_score_)
```

■ OUTPUT

```
{'alpha': 0.1}  
-35.62739011281295
```

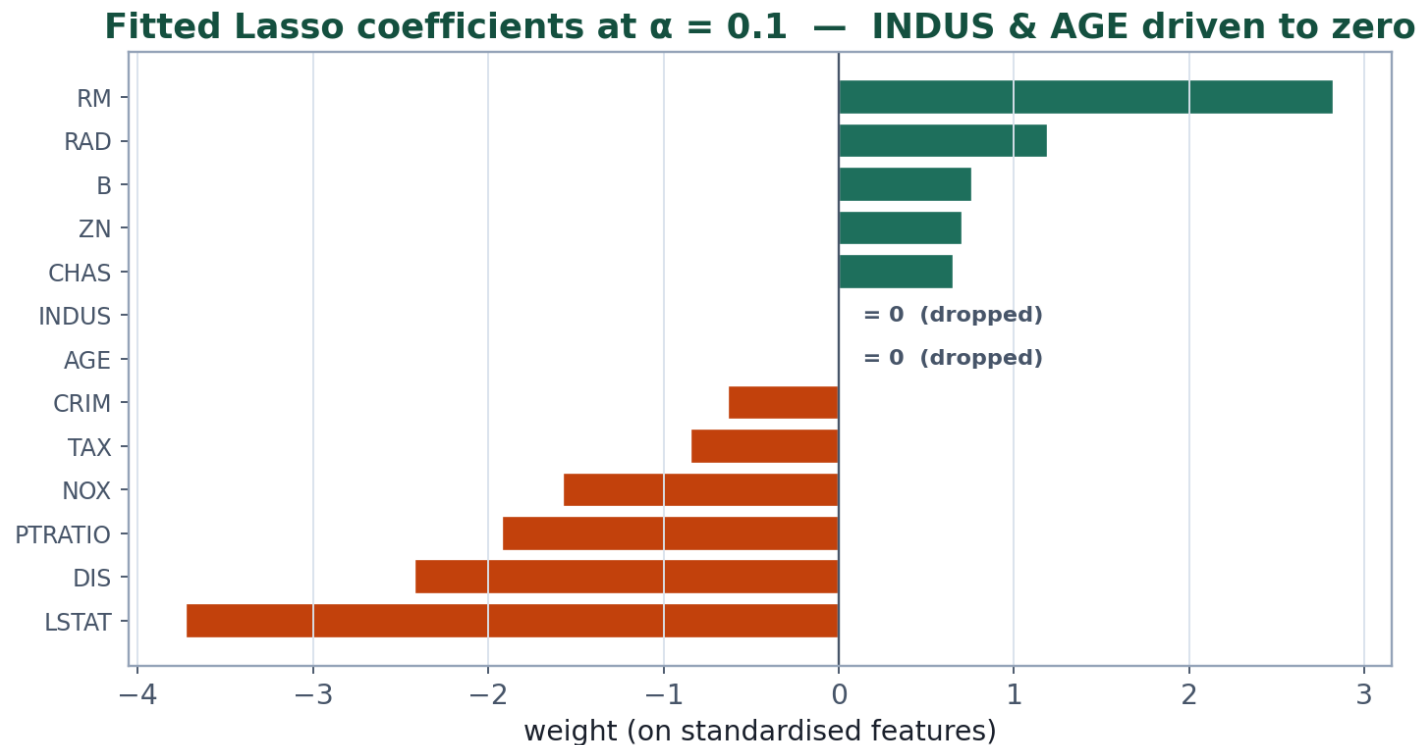
Reading the result

- **best_params_** → winning alpha (here **0.1**).
- **best_score_** → negative MSE (closer to 0 = better).
- Inspect `best_estimator_.coef_` to see which weights are 0.

RESULT

Feature selection in action

At the best alpha, Lasso drove **INDUS** and **AGE** to exactly zero — dropping them automatically. The strongest drivers are **RM** (rooms, +) and **LSTAT** (lower-status %, -).



REFERENCE

The full script

```
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV

boston = fetch_openml(name="boston", version=1, as_frame=True, parser="pandas")
df = boston.frame
df["Price"] = boston.target

X = df.drop(columns=["MEDV", "Price"])
Y = df["Price"]
X = StandardScaler().fit_transform(X)    # Lasso is scale-sensitive

lasso = Lasso(max_iter=10000)
params = {"alpha": [1e-3, 1e-2, 0.1, 1, 5, 10, 20]}
lasso_cv = GridSearchCV(lasso, params, scoring="neg_mean_squared_error", cv=5)
lasso_cv.fit(X, Y)

print(lasso_cv.best_params_)    # {'alpha': 0.1}
print(lasso_cv.best_score_)    # -35.627...
```

RECAP

Lasso in one minute

- Lasso = linear regression + **L1 penalty** ($\alpha \sum |w|$).
- It shrinks weights **and sets weak ones to exactly 0** → feature selection.
- **Always standardise** features first; tune alpha with cross-validation.
- Best when you have **many features but only a few matter**.
- On Boston data: best alpha **0.1**, and it dropped **INDUS & AGE**.